

Divide and Conquer

Analysis and Design of Algorithms



The Core Idea

01

Divide: Divide the given problem into sub-problems using recursion.

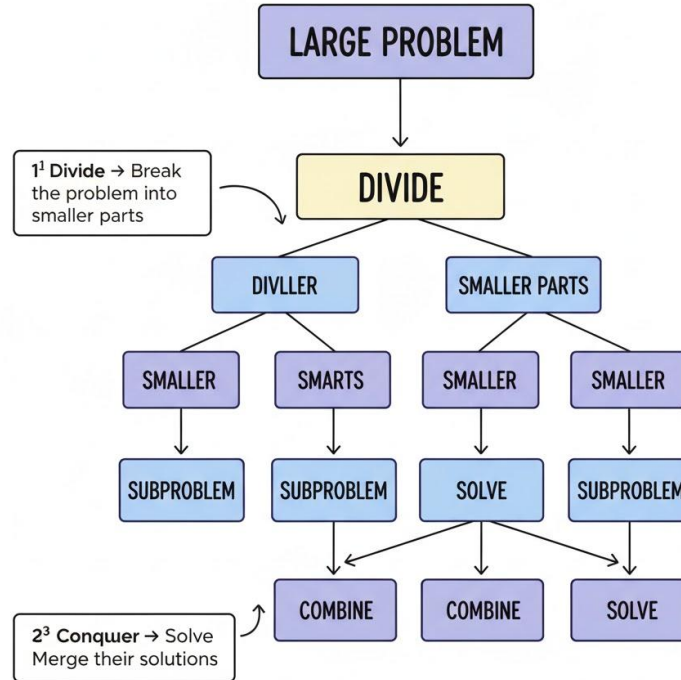
02

Conquer: Solve the smaller sub-problems recursively. If the subproblem is small enough, then solve it directly.

03

Combine: Combine the solutions of the sub-problems that are part of the recursive process to solve the actual problem.

The Core Idea



Merge Sort

- `arr = [42, 17, 8, 23, 56, 14]`



Step 1 : Divide

- Left: [42, 17, 8]
- Right: [23, 56, 14]



Step 2 : Divide Left Part

- Left: [42, 17]
- Right: [8]

[42, 17] $\rightarrow$ split $\rightarrow$ [42], [17]



Step 3 : Merge [42] and [17]

Compare 42 vs 17 → place 17 first → then 42

[17, 42]

- Merge with [8]:

Compare 17 vs 8 → place 8 first

Then 17, 42

[8, 17, 42]



Step 2 : Divide Right Part

- Right = [23, 56, 14]
 Left: [23, 56]
 Right: [14]



Step 2 : Divide Right Part

- Right = [23, 56, 14]

Left: [23, 56]

Right: [14]

- First $\rightarrow$ [23, 56] $\rightarrow$ [23], [56]
- Compare, Merge $\rightarrow$ [23, 56]



Step 2 : Divide Right Part

- Right = [23, 56, 14]
 Left: [23, 56]
 Right: [14]
- First $\rightarrow$ [23, 56] $\rightarrow$ [23], [56]
- Compare, Merge $\rightarrow$ [23, 56]
- Merge with [14]:
- Compare 23 vs 14 $\rightarrow$ place 14 first, then 23, 56
 Right part sorted: [14, 23, 56]



Step 5: Merge Left and Right

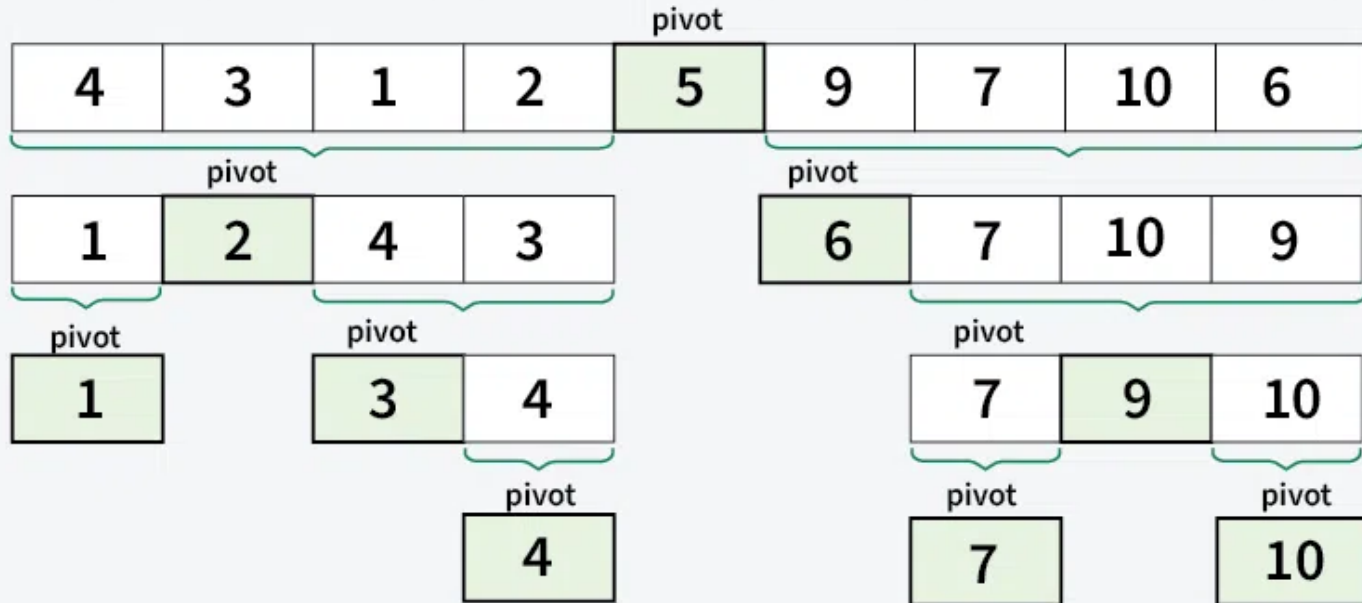
- Compare [8, 17, 42] and [14, 23, 56]:

[8, 14, 17, 23, 42, 56] Final



Quick Sort

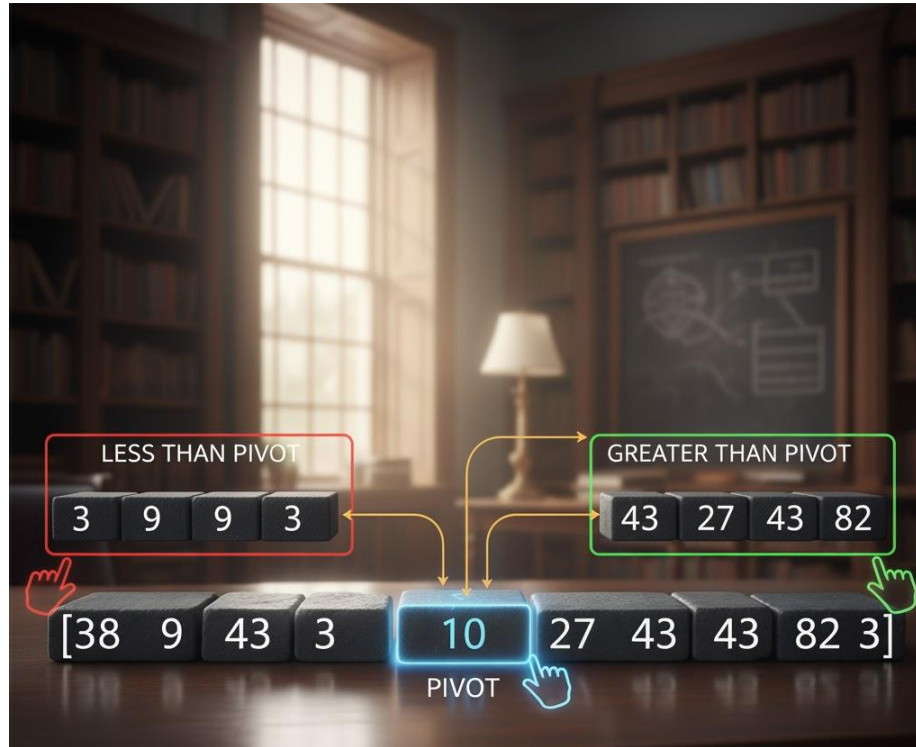
Here, we have represented the recursive call after each partitioning step of the array.



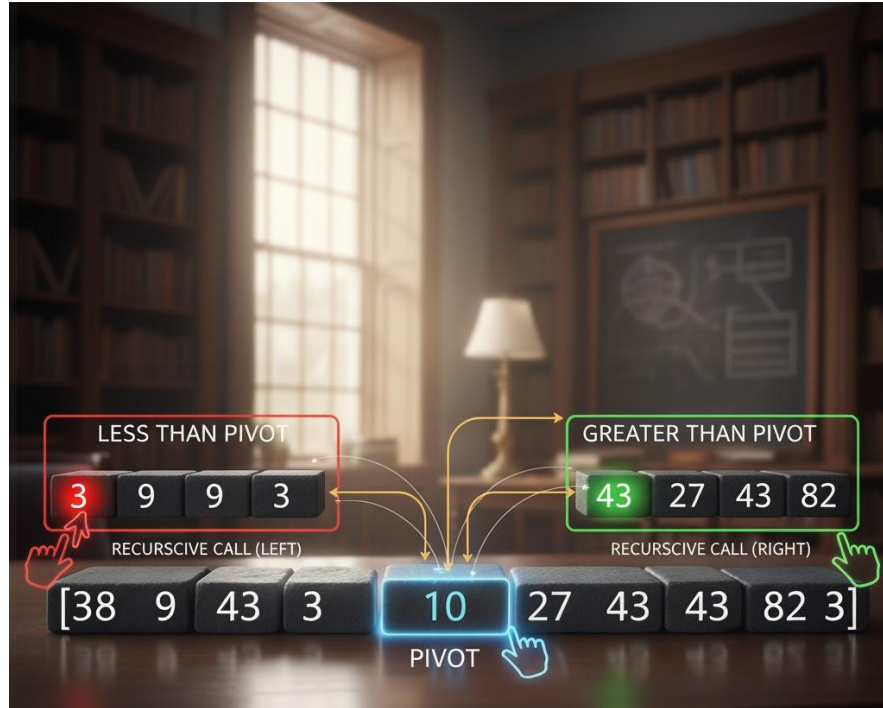
Quick Sort Step 1: Choose a pivot



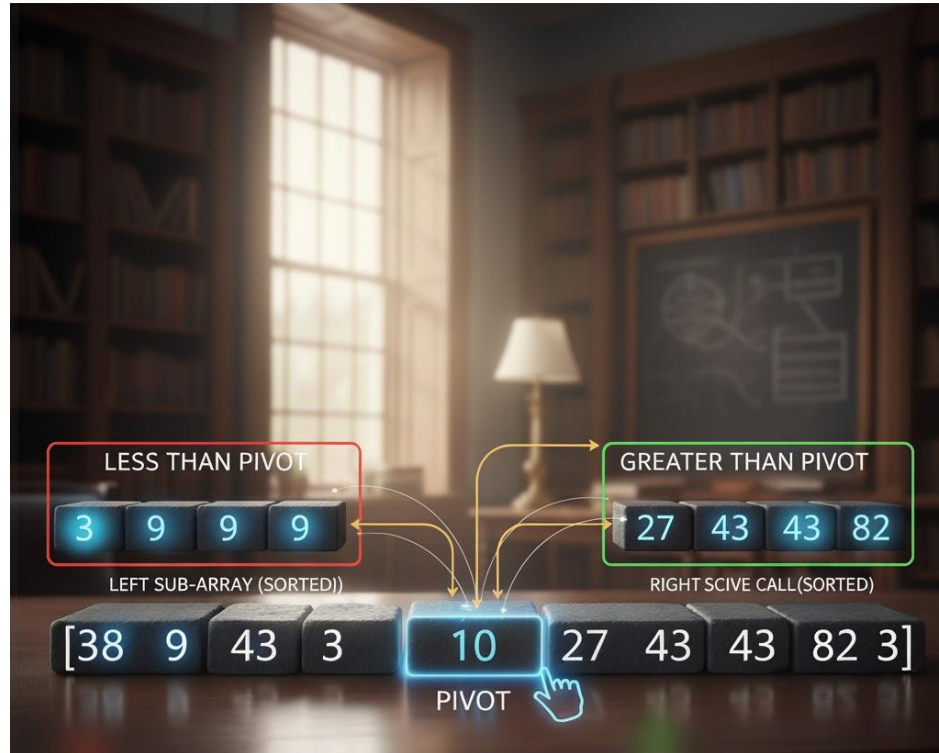
Quick Sort Step 2: Partition the array



Quick Sort Step 3: Recursively Call



Quick Sort Step 4: Base Case



Example 1

Array: [29, 10, 14, 37, 13]

Pivot = 13



Example 1 : Partition

Index	Value	Action
0	29	$29 > 13 \rightarrow$ no swap
1	10	$10 \leq 13 \rightarrow$ swap with $\text{arr}[i+1] \rightarrow$ $\text{arr} = [10, 29, 14, 37, 13]$
2	14	$14 > 13 \rightarrow$ no swap
3	37	$37 > 13 \rightarrow$ no swap



Example 1: Recursive Calls

- Array after partitioning
[10, 13, 14, 37, 29]
- Left partition: [10] → single element → done
- Right partition: [14, 37, 29] → continue sorting



Example 1: Right Partition

- Array: [14, 37, 29]
- quicksort(arr, 2, 4)
- Pivot = 29



Example 1: Right Partition

Index	Value	Action
2	14	$14 \leq 29 \rightarrow$ swap (no change)
3	37	$37 > 29 \rightarrow$ no swap



Example 1: Recursive Calls

- Array after Right partitioning
[10, 13, 14, 29, 37]



Example 2

Array = [45, 23, 89, 12, 67, 34]

Pivot = 34



Example 2

index	value	Action
0	45	$45 > 34 \rightarrow$ no swap
1	23	$23 \leq 34 \rightarrow$ swap with $\text{arr}[i+1] \rightarrow$ $\text{arr} = [23, 45, 89, 12, 67, 34]$
2	89	$89 > 34 \rightarrow$ no swap
3	12	$12 \leq 34 \rightarrow$ swap with $\text{arr}[i+1] \rightarrow$ $\text{arr} = [23, 12, 89, 45, 67, 34]$
4	67	$67 > 34 \rightarrow$ no swap



Example 2: Recursive Calls

- Array after partitioning
arr = [23, 12, 34, 45, 67, 89]
pivot_index = 2
- Left partition: [23, 12] → sort next
- Right partition: [45, 67, 89] → sort next



Example 2: Left Partition

- Left Partition [23, 12]
- Pivot = 12

Compare 23 $\rightarrow$ 23 > 12 $\rightarrow$ no swap

Swap pivot with arr[i+1] $\rightarrow$ swap 12 and 23:

[12, 23, 34, 45, 67, 89]



Example 2: Right Partition

- **Right Partition [45, 67, 89]**

Pivot = 89

- $45 \leq 89 \rightarrow$ swap (no change)
- $67 \leq 89 \rightarrow$ swap (no change)

Swap pivot with arr[i+1] $\rightarrow$ swap 89 with 89 $\rightarrow$ no change:

[12, 23, 34, 45, 67, 89] Final Sorted Array

